

Qualifying the HTTP DSL

A reference for Okapi's module layout and its qualify-everything convention

Okapi

July 18, 2026

Introduction

Okapi's HTTP codec DSL is spread across roughly twenty modules — a core combinator engine, shared `Headers/Body` codecs, per-side `Request/Response` facades, a full RFC 9651 Structured Field Values implementation, and the value-level types each side's parser/printer produce and consume. This document is a reference: what each import gives you, why everything is designed to be qualified, and a worked example putting it together.

The rule the whole design follows: every name is qualified, always, including field names inside record-update syntax. This isn't a style preference — it's load-bearing. Several types this DSL exposes genuinely share field names and even type names across modules (`Request`'s `path/headers/body` versus `Response`'s `headers/body`, or `Okapi.Request.Data`'s `Request` versus `Okapi.HTTP.Request`'s own `Request`). A qualified-only import never brings its fields into unqualified scope, so it can never contribute a false candidate to GHC's record-update disambiguation — that's the actual mechanism qualifying everything buys you, not just readability (Part 3).

Every claim here was checked against the real library — the worked example at the end is real, compiling code, verified against `lib/src` with GHCi. Generic-deriving (`Path.derived`, `GPath/GQuery/GHeaders`, etc.) is out of scope — it has its own qualification story.

Part 1 — The three-layer architecture

Layer	Modules	What it's for
Okapi.HTTP.*	Tree, Structured.*, Headers, Body, HTTP (incl. Morph/morph), <code>Request(.Method/.Path/.Query/.Headers/.Headers.Cookie/.Body),</code> <code>Response(.Status/.Headers/.Headers.SetCookie/.Headers.Attributes/.Body)</code>	Describing and coding one HTTP message.
Okapi.Request.* / Okapi.Response.*	<code>Request.Data/.Result/.Failure</code> , <code>Response.Data/.Result/.Failure</code>	This value-level contains parts parser/printer produce and consume. No combinators of their own.
Okapi.Server/.Client/ plus the bare Okapi facade	<code>Server/.Function/.Link/.OpenApi</code> Function, Link, OpenApi	Consumes an HTTP contract, produces something else — a server, a client function, an OpenAPI document, hyperlinks.

`Okapi.HTTP.Tree` is the foundation the other layers build on: the generic applicative combinator engine that `field`, `json`, `seg`, and every other leaf combinator desugars through via `ApplicativeDo`. `Okapi.HTTP.Structured.*` (RFC 9651) sits alongside it, equally generic. `Morph` — the newtype lifting a shape-only type (`HTTP`, `Okapi.Link.Link`, `Okapi.Client.Client`) into the 2-arg slot a heterogeneous HKD record needs — lives in `Okapi.HTTP` next to `Signature`, the thing it exists to carry.

`Okapi.Request/Okapi.Response`'s `Data/Result/Failure` leaves sit at the bare top level, not nested under `Okapi.HTTP`: they aren't DSL machinery or artifacts consumed *from* an HTTP contract, they're the value-level twin of `Request/Response`, grouped side-first like `Okapi.HTTP.Request.Method/.Path/etc.` (`Okapi.Request.Data` exports a type named `Request`, not `Data` — Part 4). `Okapi.Server/.Client/.Function/.Link/.OpenApi` share the same shape (each takes an HTTP contract, produces something else); the bare `Okapi` facade re-exports it all unqualified.

Part 2 — What each qualifier gives you

One documented, correct path per operation — nothing is re-exported across two facades for import-count convenience; every combinator lives at exactly one qualifier, its true defining module.

Okapi (bare, unqualified — safe, none of it is Request-or-Response-specific): the HTTP contract type `((:->)/(:-<))`, `Morph/morph`, `Responses/responses/getResponses/parseResponses/printResponses`, the whole `Server/Handle/Client/Link` layer, and the generic `Tree` surface

(SymTree, Leaf(..), HasLeaf(..), (=.), leaf combinators int/integer/bool/text/uuid/etc.).
Not here: Request/ Response, base/get/post/etc., method/status singletons,
seg/param/field/json/cookie/setCookie.

Okapi.HTTP (qualified HTTP): just the contract type’s own machinery —
Signature, Base, the unified (:&) field tags (covering Signature, Request,
and Response alike), annotate/stripTags/ collectTags.

Okapi.HTTP.Headers/Okapi.HTTP.Body (qualified Headers./Body): the
genuinely side-free operations — field/field'/field_/ contentType/fieldStruct/fieldBareItem/fieldItem
fieldDict/MediaType(..) (Headers), json/jsonValue/none/ None(..)/IsoJson
(Body) — free in the phantom ForRequest/ ForResponse tag, one binding for
either side, reached only here.

Okapi.HTTP.Request (qualified Req.): Request(..), base (the maximally
generic start), get/post/put/delete/patch/head/ options/connect/trace,
narrowing functions method/path/query/ headers/body, seg/seg_/lit/secs,
param/param'/param_/flag/flag'/list/list'/ArrayStyle(..), and the
side-pinned cookie/cookie'/form. get/post/.../head deliberately shadow
Prelude when qualified as Req. (head collides even unqualified in its own
defining module, hidden explicitly, same as Method.hs/Status.hs hide
Prelude.print). Method singletons (GET/Get/etc.) reach via Method.
directly (Part 5), not Req..

Okapi.HTTP.Response (qualified Res.): Response(..), base, all 47 status-
named response codecs (continue .. ok .. notFound .. networkAuthenticationRequired,
full list in Part 4), headers/body narrowing, and the side-pinned setCookie.
Status singletons and all 9 Attributes combinators reach via Status./Attr.
directly (Part 5).

A module building only one side already needs Req./Res. plus Headers./Body.
for anything header- or body-shaped — the real cost of every name meaning
exactly one thing regardless of which qualifier reaches it.

Okapi.Request.Data/.Result/.Failure, Okapi.Response.Data/.Result/.Failure:
the decoded value, the intermediate per-field parse, and the accumulated per-
field error, one leaf module per side per kind. Reached with their own qualified
import, distinct from Req./ Res..

Part 3 — The qualification mechanism

Request and Response share field names (headers, body); Okapi.Request.Data’s
Request shares a type *name* with Okapi.HTTP.Request’s Request. Neither is
a problem, because of one mechanism, applied consistently: **a qualified-only
import never brings its fields into unqualified scope**, so it can never
contribute a candidate to GHC’s record-update field-set resolution.

Same-module case. With Req/Res qualified-only, a *bare* field name in an

update is never in scope, whether or not it's unique — GHC reports “not in scope,” not “ambiguous,” with a suggested-fix listing every qualified candidate:

```
Req.base { path = reviewPath, headers = ... }
-- error: Not in scope: record field 'path'
-- Suggested fix: Perhaps use record field of Request 'Req.path'

Res.ok { headers = reviewOkHeaders, body = Body.json @Review }
-- error: Not in scope: record field 'headers'
-- Suggested fix: Perhaps use one of these:
-- record field of Request 'Req.headers', record field of Response 'Res.headers'
```

Same error class either way — only the candidate count differs. The fix is identical: qualify the field name, keeping record-update syntax (or use narrowing functions instead):

```
reviewOk = Res.ok { Res.headers = reviewOkHeaders, Res.body = Body.json @Review }
-- equivalently: Res.body (Body.json @Review) (Res.headers reviewOkHeaders Res.ok)
```

A genuinely *ambiguous* update (GHC can't even suggest a fix) only happens if a type's fields are *also* brought into unqualified scope some other way — not something the qualify-only convention hits.

Cross-module case. The same rule applies when two *different* modules share a field name — e.g. `Okapi.Link`'s `URI { path :: Text, query :: Text }` versus `Request`'s own `path`. With both imported qualified-only, a bare-field update isn't even ambiguous, just not in scope, and GHC's suggestion points straight at the fix:

```
import Okapi.HTTP.Request qualified as Req
import Okapi.Link qualified as Link

test = Req.delete { path = deleteReviewPath }
-- error: Not in scope: record field 'path'
-- Suggested fix: Perhaps use one of these:
-- record field of URI 'Link.path' (imported from Okapi.Link),
-- record field of Request 'Req.path' (imported from Okapi.HTTP.Request)
```

Importing the *other* module unqualified breaks the convention and hides the problem differently — the bare field silently resolves to whichever type's fields are unqualified-visible, failing as a type mismatch (`Couldn't match expected type 'Req.Request ...' with actual type 'URI'`) instead of a clean scope error.

The practical rule, unconditionally: qualify the field name in every record update, `Req.base { Req.path = ... }`, not just when GHC complains. The same “qualification decides scope” logic recurs elsewhere in the DSL: `param/flag/list` mean different things at different levels (query-string parameters, RFC 9651 item parameters, `Set-Cookie` flags), and each needs its own qualifier the moment a function touches more than one.

Part 4 — Naming conventions

1. **Types and constructors stay fully spelled out.** `Request`, `Response`, `Headers`, `Structured`, `Attributes`, `Dictionary`, `Parameters`, `KnownMethod`, `KnownStatus`.
2. **Self-titling, two ways.** A DSL leaf module's own name is also its exported type's name: `Okapi.HTTP.Request.Method` exports `Method` (one type per kind, side-independent). `Data/Result/Failure` title by *side* instead: `Okapi.Request.Data` exports `Request`, not `Data` — it's "a Request, decoded," not "a Data."
3. **Term-level combinators abbreviate a recognized short form when the full word is long enough to matter** (`segment/segment_/segments` → `seg/seg_/segs`, `attribute/attribute'` → `attr/attr'`, `dictionary` → `dict`), and stay as-is otherwise (`field`, `param`, `flag`, `list`, `item`, `member`, `status`, `method`, `base`, `raw`, `json`, `none`, `form`). `field_/param_/seg_/bareItem_` all use the same underscore suffix for "assert a fixed, known value." `base` is the unconstrained smart constructor wherever a `Base` synonym exists; `raw` is the same role in `Structured.*/Attributes`, which have none.
4. **A name never restates its own qualifier.** `Req.base/Res.base`, not `req/res`; `Req.get/Req.post/...`, not `reqGET/reqPOST`. Names colliding with `Prelude` when unqualified (`head`) are hidden explicitly at their defining site, same as `Method.hs/Status.hs` hide `Prelude.print`.

Status codes. All 47 are spelled-out names, reused verbatim from `http-types`'s own reason-phrase-derived aliases, minus the trailing number — `Res.` qualification already disambiguates, so `Res.ok` reads better than `Res.ok200`. Reusing rather than inventing matters for correctness too: a few codes have shifted reason phrases across RFC revisions (413/414/416 in particular), and `Status.hs` already sources its reason phrases from `http-types`, so matching its names keeps everything consistent with one source of truth.

```
continue switchingProtocols ok created accepted nonAuthoritative
noContent resetContent partialContent multipleChoices movedPermanently
found seeOther notModified useProxy temporaryRedirect permanentRedirect
badRequest unauthorized paymentRequired forbidden notFound methodNotAllowed
notAcceptable proxyAuthenticationRequired requestTimeout conflict
gone lengthRequired preconditionFailed requestEntityTooLarge
requestURITooLong unsupportedMediaType requestedRangeNotSatisfiable
expectationFailed imATeapot unprocessableEntity preconditionRequired
tooManyRequests requestHeaderFieldsTooLarge internalServerError
notImplemented badGateway serviceUnavailable gatewayTimeout
httpVersionNotSupported networkAuthenticationRequired
```

Each is reachable in a type signature via the matching `SNNN` alias from `Okapi.HTTP.Response.Status: ok :: Response S200 Types.ResponseHeaders (IO LBS.ByteString)`.

`noContent` (204) is also the shared body-emptiness combinator’s name in spirit — resolved by naming *that* one `none` instead (`Okapi.HTTP.Body`, matching its own result type `None`, the same way `json/jsonValue` are named after what they produce). `Res.noContent` for 204 and `Body.none` for an empty body now both mean exactly what they say.

Part 5 — Reference: the rest

Functions that exist and are real, but don’t naturally show up in a typical contract:

Name	Module	Purpose
<code>base / parser / printer</code>	every leaf module with a <code>Base</code> synonym, plus <code>Okapi.HTTP.Tree</code>	Unconstrained pass-through (<code>raw</code> , not <code>base</code> , where no <code>Base</code> synonym exists — <code>Structured.*/Attributes</code>), and the low-level codec functions every combinator builds on.
<code>Method.GET...TRACE/.GetRequest/knownMethod</code>	<code>Req.*/Method</code>	The method singletons — not re-exported onto <code>Req.</code> (Part 2).
<code>Method.method/.parse/.Request.Method</code>	<code>Req.Method</code>	Build/inspect a <code>Method</code> codec (<code>print</code> shadows <code>Prelude.print</code> unqualified).
<code>Status.S100...S511/KnownResponse/knownStatus</code>	<code>Res.StatusCode/knownStatus</code>	The status singletons — not re-exported onto <code>Res.</code> (Part 2).
<code>Status.status/.parse/.Response.Status</code>	<code>Res.StatusCode</code>	Build/inspect a <code>Status</code> codec directly.
<code>Attr.attr/.attr'/.secureResponseHeaders/Attr.attr/.flag</code>	<code>Res.Headers/Attr.attr/.flag</code>	combinators — none curated onto <code>Res.</code> (Part 2).
<code>Struct.item/.list/.dict/Okapi.HTTP.Structured</code>	<code>Okapi.HTTP.Structured</code>	The escape hatch <code>fieldStruct</code> sugars over.
<code>BareItem.displayString\$bytes\$seq/BareItem</code>	<code>ByteSeq/BareItem</code>	RFC 9651 §3.3.8/§3.3.5 — Display String and Byte Sequence.
<code>knownMethodToStd/knownMethodToStd</code>	<code>StdToStd</code>	Convert to/extract from <code>http-types</code> .

Part 6 — Worked example

A three-endpoint “product reviews” API, compiling as-is against this library. Endpoint 1 is the dense one, using every combinator family (`BareItem`, `Item`, `Dictionary`, `List`, `Set-Cookie`) across its path, query, headers, and response. Endpoint 2 exists mainly for `segs`. Endpoint 3 is the `Responses` demo, including the `path/URI` fix from Part 3. A module building only one side still needs its facade plus `Headers/Body` for anything header- or body-shaped — the deliberate tradeoff this design makes (Part 2).

```
{-# LANGUAGE ApplicativeDo #-}
{-# LANGUAGE BlockArguments #-}
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE DeriveAnyClass #-}
{-# LANGUAGE DeriveGeneric #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE TypeApplications #-}

module MaxQual where

import Data.Aeson qualified as Aeson
import Data.Function ((&))
import Data.OpenApi (ToSchema)
import Data.Text (Text)
import Data.UUID (UUID)
import Data.ByteString (ByteString)
import Network.HTTP.Types qualified as Types
import Data.Maybe (fromMaybe)
import GHC.Generics (Generic)

import Okapi -- generic mode layer + Tree engine; safe to leave bare
import Okapi.HTTP.Request qualified as Req -- everything Request-specific
import Okapi.HTTP.Response qualified as Res -- everything Response-specific
import Okapi.HTTP.Request.Method qualified as Method -- the smart constructor itself
import Okapi.HTTP.Request.Query qualified as Query -- ArrayStyle constructors
import Okapi.HTTP.Headers qualified as Headers -- shared header combinators, their true home
import Okapi.HTTP.Body qualified as Body -- shared body combinators, their true home
import Okapi.HTTP.Response.Status qualified as Status -- status singletons, their true home
import Okapi.HTTP.Response.Headers.Attributes qualified as Attr -- all 9 Attributes combinators
-- Structured Field Values (RFC 9651) -- fully generic, no Req./Res. needed.
import Okapi.HTTP.Structured qualified as Struct
import Okapi.HTTP.Structured.BareItem qualified as BareItem
import Okapi.HTTP.Structured.Item qualified as Item
import Okapi.HTTP.Structured.List qualified as SList
import Okapi.HTTP.Structured.Dictionary qualified as Dict
import Okapi.HTTP.Structured.Parameters qualified as Params
```

```

import Network.Wai qualified as Wai -- for the parseResponses/printResponses demo
import Okapi.Response.Data qualified as Data
data Review = Review
  { reviewId :: Int
  , stars    :: Int
  , comment  :: Text
  }
  deriving (Generic, Show, Aeson.FromJSON, Aeson.ToJSON, ToSchema)

clientToken :: BareItem.Token
clientToken = fromMaybe (error "bad literal token") (BareItem.mkToken "okapi-v2")

-- Endpoint 1: GET /v2/products/{productId}/reviews/{reviewId} -- `&` chains
-- narrowing functions left to right; projection helpers (`hp1` etc.) are a leading `let`.
getReview = Req.base
  & Req.path do
    Req.seg_ int 2
    Req.lit "products"
    pid <- fst =. Req.seg "productId" uuid
    Req.lit "reviews"
    rid <- snd =. Req.seg "reviewId" int
    pure (pid, rid)
  & Req.query do
    let qp1 (a,_,_,_) = a
        qp2 (_,b,_,_) = b
        qp3 (_,_,c,_) = c
        qp4 (_,_,_,d) = d
    verbose <- qp1 =. Req.flag' "verbose"
    limit <- qp2 =. Req.param' "limit" int
    tagFilter <- qp3 =. Req.list' Query.Exploded "tag" text
    Req.param_ "api" int 2
    fixedFmt <- qp4 =. Req.param "format" text
    pure (verbose, limit, tagFilter, fixedFmt)
  & Req.headers do
    let hp1 (a,_,_,_,_,_) = a
        hp2 (_,b,_,_,_,_) = b
        hp3 (_,_,c,_,_,_) = c
        hp4 (_,_,_,d,_,_) = d
        hp5 (_,_,_,_,e,_) = e
        hp6 (_,_,_,_,_,f) = f
    Headers.field_ "x-service" "reviews" -- field-family combinators are `Headers.`; on
    Headers.contentType Headers.JSON
    apiKey <- hp1 =. Headers.fieldBareItem "x-api-key" (BareItem.token :: Leaf BareItem)
    buildNum <- hp2 =. Headers.fieldItem "x-client" do -- fixed marker token + a real p
      Item.bareItem_ (BareItem.unToken clientToken)
      n <- Item.params (Params.param "build" integer)

```



```

    pure n
    traceId <- hp3 =. Headers.field' "x-trace-id" uuid
    prefs <- hp4 =. Headers.fieldDict "prefer" do
      a <- fst =. Dict.member "compact" (Item.bareItem bool)
      b <- snd =. Dict.member' "notes" (Item.bareItem bool)
    pure (a, b)
    sid <- hp5 =. Req.cookie' "sid" uuid
    lang <- hp6 =. Req.cookie "lang" text
    pure (apiKey, buildNum, traceId, prefs, sid, lang)
  & Req.body Body.none
-- Endpoint 1's responses -- List-shaped headers: a flat list of scalars,
-- and the general innerListOf heterogeneous-composition shape (§3.1.1).
cacheTags = SList.items (Item.bareItem text)
batchGroups = do
  a <- fst =. SList.innerListOf
    (SList.innerItem (Item.bareItem text))
    (Params.param "lvl" integer)
  b <- snd =. SList.innerListOf
    (SList.innerItems (Item.bareItem integer))
    (Params.param' "lvl" integer)
  pure (a, b)
-- Same `&`/`BlockArguments` treatment, reusing the same `Headers.` functions.
reviewOk = Res.ok
  & Res.headers do
    let rp1 _ = ()
      rp2 (a,_,_,_,_) = a
      rp3 (_,b,_,_,_) = b
      rp4 (_,_,c,_,_) = c
      rp5 (_,_,_,d,_) = d
      rp6 (_,_,_,_,e) = e
    rp1 =. Headers.contentType Headers.JSON
    limit <- rp2 =. Headers.field "x-ratelimit-limit" int
    rateItem <- rp3 =. Headers.fieldItem "x-ratelimit" do
      n <- fst =. Item.bareItem integer
      params <- snd =. Item.params do
        w <- fst =. Params.param "window" integer
        b <- snd =. Params.flag' "burst"
      pure (w, b)
    pure (n, params)
    tags <- rp4 =. Headers.fieldList "cache-tags" cacheTags
    groups <- rp5 =. Headers.fieldList "x-batch-groups" batchGroups
    session <- rp6 =. Res.setCookie "session" uuid do
      let ap1 (a,_,_,_) = a
        ap2 (_,b,_,_) = b
        ap3 (_,_,c,_) = c
        ap4 _ = ()

```

```

        ap5 (_,_,_,d) = d
        ap6 _ = ()
age <- ap1 =. Attr.maxAge
dom <- ap2 =. Attr.domain
pth <- ap3 =. Attr.path
ap4 =. Attr.secure
sameSite <- ap5 =. Attr.attr "SameSite" (leaf :: Leaf Attr.Attributes ByteString)
ap6 =. Attr.flag "Partitioned"
pure (age, dom, pth, sameSite)
pure (limit, rateItem, tags, groups, session)
& Res.body (Body.json @Review)
getReviewContract = getReview :-> reviewOk
-- Endpoint 2: GET /products/{productId}/reviews/tags/{tag}+ -- `segs`, `Req.method Method.(
listReviews = Req.base
    & Req.method Method.Get
    & Req.path do
        Req.lit "products"
        pid <- fst =. Req.seg "productId" uuid
        Req.lit "reviews"
        Req.lit "tags"
        tags <- snd =. Req.segs text
        pure (pid, tags)
    & Req.body Body.none
listOk = Res.ok & Res.body (Body.json @[Review])
listReviewsContract = listReviews :-> listOk

-- Endpoint 3: a `Responses` contract, two response alternatives. Uses the
-- `&-pipe` fix for the `path`/`URI` collision (Part 3).
deleteReviewReq = Req.delete
    & Req.path do
        Req.lit "products"
        pid <- fst =. Req.seg "productId" uuid
        Req.lit "reviews"
        rid <- snd =. Req.seg "reviewId" int
        pure (pid, rid)
deletedRes = Res.noContent
notFoundRes = Res.notFound & Res.body (Body.json @Text)
data DeleteReviewResponses f
    = Deleted (f Status.S204 Types.ResponseHeaders (IO Body.None))
    | ReviewNotFound (f Status.S404 Types.ResponseHeaders (IO Text))
    deriving (Generic, Responses)
deleteReviewResponses = responses @DeleteReviewResponses deletedRes notFoundRes
deleteReviewContract = deleteReviewReq :-< deleteReviewResponses
demoPrintDeleted :: IO Wai.Response
demoPrintDeleted = printResponses deleteReviewResponses (Deleted (Data.Response Status.S204
demoParseIncoming waiResponse = parseResponses deleteReviewResponses waiResponse

```